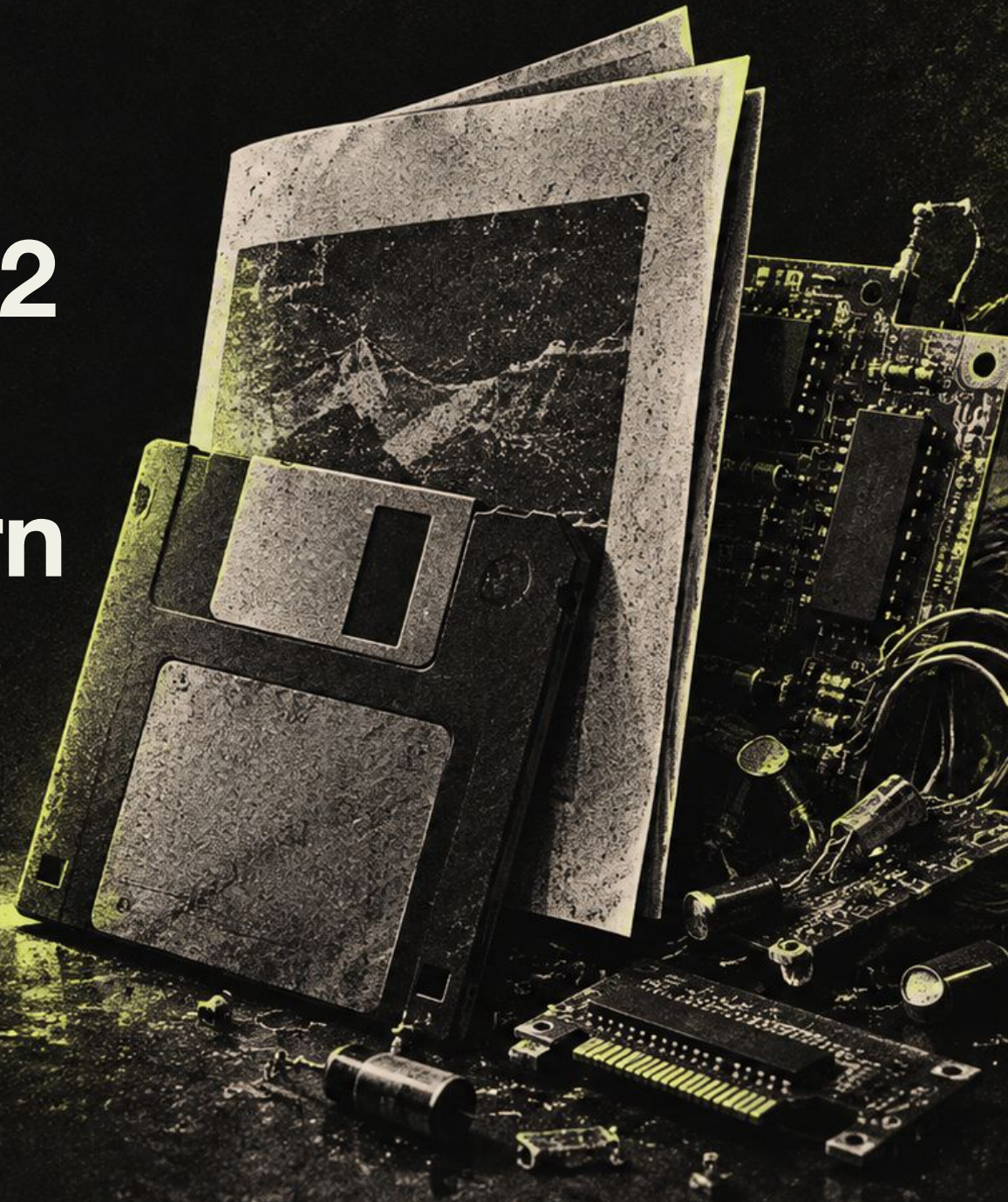


Reconstructing a 1992 Virus to Understand Modern Malware

Charley Pfaff

GrrCON



Get the files. Watch along.

SHARED CONTENT

github.com/bl4ck0ut/grrcon

That's a zero in bl4ck0ut, not the letter O.

WATCH ALONG ON YOUR DEVICE

Join Wi-Fi: CHARLEY-LIVE Open: charley.live

No Internet connection required.

This will break. Highly experimental in every way...

A day at the DOS prompt

**DOS. A drive.
Oregon Trail.**

Work in DOS
Mount the lab drive
Play Oregon Trail, woot!
Observe the result



Cousin!!

Why do this?

- What happened back then? Vienna and Cascade infected .COM files.
- How hard would rebuilding one really be? Shout out to the OG coders.
- Start with a known public sample: the 1992 printed listing.
- Learn from the past by inspecting what the code actually does.
- For Fun!.
- This work was done over eight months ago.

History repeats itself. I wanted to single-step it.

What you will walk away with

- Historical knowledge of DOS .COM file infectors
- A DOS 6.22 environment with networking enabled. Woot!
- Assembly source files for testing in the lab
- Disinfector code for this variant

The start: image to code

[illegible]

Spring 1992 2600 Magazine Page 5

[illegible]

Page 6 2600 Magazine Spring 1992

Spring 1992

Pages 5–6

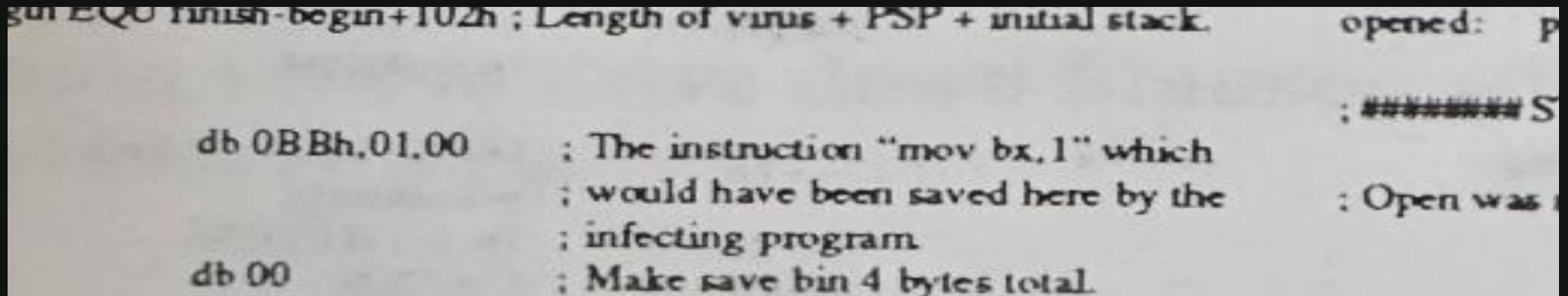
DOS .COM listing

Tools needed

- A hypervisor with virtual floppy-drive support
- A DOS environment for running the programs
- Assemblers: NASM and MASM
- DOS DEBUG to inspect memory and instructions
- A way to move source files and binaries into the lab

Edit on the host. Transfer to the lab. Inspect in DOS.

One character changed the instruction



```
gnt EQU finish-begin+102h ; Length of virus + PSP + initial stack.      opened:  p
; ##### S
db 0BBh,01,00      ; The instruction "mov bx,1" which
                  ; would have been saved here by the      ; Open was
                  ; infecting program.
db 00              ; Make save bin 4 bytes total.
```

PHOTO

0BBh, 01, 00

FIRST TRANSCRIPT

08Bh, 01, 00

Source photograph above. Initial transcription below right.

NASM conversion needed byte checks

SAME VALUE

0FF0Eh

0xFF0E

DIFFERENT INSTRUCTIONS

```
mov cx, 0
```

```
xor cx, cx
```

Equivalent register values can still hide different bytes and side effects.

The debugger made the bytes visible

```
11 file(s)          540,272 bytes
                   512,000 bytes free

A:\>debug find2.com
-d 100 1100
111C:0100  0E 1F BA 5D 01 B4 1A CD-21 BA 4D 01 31 C9 B4 4E  ...J....!.M.1..N
111C:0110  CD 21 72 34 BA 53 01 31-C9 B4 3C CD 21 72 29 89  .!r4.S.1..<.!r).
111C:0120  C3 BE 7B 01 BF 88 01 AC-AA 84 C0 75 FA C6 45 FF  ..{.....u..E.
111C:0130  0D C6 05 0A 47 C6 05 00-BA 88 01 89 F9 29 D1 49  ....G.....).I
111C:0140  B4 40 CD 21 B4 3E CD 21-B8 00 4C CD 21 2A 2E 43  .@.!.>.!..L.!*.C
111C:0150  4F 4D 00 46 4F 55 4E 44-2E 54 58 54 00 00 00 00  OM.FOUND.TXT....
111C:0160  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
111C:0170  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
111C:0180  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
111C:0190  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
111C:01A0  00 00 00 00 00 00 00 00-41 AF 21 13 41 AF 21 13  ....A.!.A.!.
111C:01B0  41 04 22 13 41 04 22 13-41 2F 21 13 41 2F 21 6F  A.".A."A/!.A/!o
111C:01C0  44 60 20 EA 40 7D 22 DE-42 D9 21 DE 42 D9 21 DE  D' .@}"B.!.B.!.
111C:01D0  42 D9 21 DE 42 D9 21 DE-42 D9 21 DE 42 D9 21 DE  B.!.B.!.B.!.B.!.
111C:01E0  42 D9 21 DE 42 D9 21 23-41 D9 21 23 41 D9 21 23  B.!.B.!.#A.!.#A.!.#
111C:01F0  41 D9 21 23 41 D9 21 23-41 D9 21 23 41 D9 21 23  A.!.#A.!.#A.!.#A.!.#
```

January 19: DEBUG inspecting find2.com

Progress messages narrowed the search

```
main A:\>1
RM.
GOB ST F1 01 Found a file
    gen debug
    CI FO OP open was successful
    03 04 05 06 PA SK PL no file found payload run
```

“Found a file” and “open was successful”

Repeated traces exposed more behavior

```
A:\>
```

```
US GB FF FO FH RD IN WJ WO WU CL NX FO RE EX GB FF FO FH RD CL NX FO RE EX GB FF  
NF RE EX GB FF NF RE EX G FF NF RE EX G FF NF RE EX G FF NF RE EX G FF NF RE EX  
G FF NF RE EX G FF NF RE EX G FF NF RE EX G FF NF RE EX G FF NF RE EX @ G FF NF  
RE EX G FF F RE EX Ç G FF F RE EX G FF F RE EX G FF F RE EX G FF F RE EX G FF  
F RE EX G FF F RE EX ÷ G FF F RE EX † G FF F RE EX ⊖ G FF F RE EX † G FF F RE E  
X G FF F RE EX G FF F RE EX G FF RE EX N G FFRE EX G FF
```

January 30: repeated markers and corrupted-looking output

The later variant changes the entry path

ENTRY

Redirects execution into appended code

SAVED STATE

Retains the original entry bytes

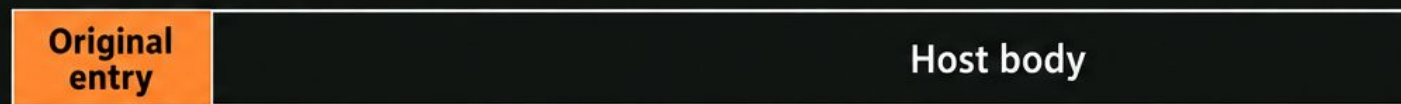
HOST RETURN

Restores those bytes in memory

How the DOS .COM variant redirects execution

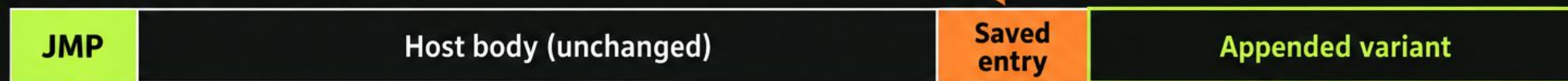
Source reading of v3.asm • Charley Pfaff

BEFORE — FILE ON DISK



Copy entry bytes

AFTER — FILE ON DISK



Execution begins here, then jumps to appended code

The host body stays in place. The file grows at the end.

WHEN THE MODIFIED FILE RUNS



FILE-CREATION PAYLOAD IN MY OTHER VERSIONS

1992c.asm / cc.asm: attempt to create empty A.TXT, close it, then exit to DOS.
payload.asm contains the standalone test. This payload is absent from v3.asm.

If A.TXT already exists, successful creation truncates it.

Host return restores memory, not the file on disk. disinfect.asm attempts on-disk repair.

Conceptual layout, not to scale. Static source analysis, not a verified runtime result.

This report showed zero detections

The screenshot shows a web interface for file analysis. At the top, a search bar contains the file hash: `cccb27478c595c14c300916f735554624ee34f2e3580bf7c7860035f49cc8cf`. Below the search bar, a large green circle displays the number `0` with `/ 61` underneath, indicating a community score. To the right, a message states: "No security vendors flagged this file as malicious". Below this, the file details are shown: `cccb27478c595c14c300916f735554624ee34f2e3580bf7c7860035f49cc8cf`, size `412 B`, last analysis date `9 days ago`, and the file type `GZIP`. The interface includes tabs for `DETECTION`, `DETAILS`, `RELATIONS`, and `COMMUNITY`. A banner encourages joining the community. Below, a table titled "Security vendors' analysis" shows results from 16 vendors, all marked as "Undetected".

Security vendors' analysis			
Acronis (Static ML)	✓ Undetected	AhnLab-V3	✓ Undetected
AliCloud	✓ Undetected	ALYac	✓ Undetected
Antiy-AVL	✓ Undetected	Arcabit	✓ Undetected
Avast	✓ Undetected	AVG	✓ Undetected
Avira (no cloud)	✓ Undetected	Baidu	✓ Undetected
BitDefender	✓ Undetected	Bkav Pro	✓ Undetected
ClamAV	✓ Undetected	CMC	✓ Undetected
CrowdStrike Falcon	✓ Undetected	CTX	✓ Undetected

1992code.com / 412 bytes / identified as gzip / 0 of 61

Another report showed 18 detections

The screenshot displays a malware analysis report for a file named 1992.EXE. At the top left, a circular gauge shows a 'Community Score' of 18 out of 62. To the right, a status bar indicates that 18 out of 62 security vendors flagged the file as malicious. The file's SHA-256 hash is e178f653f95b87cd7ed2eb5124a0d11520ecac132c1eacf22e6b7a3efee263e3, and its size is 924 B. The file type is identified as 1992.EXE with a magic number (mz) of 1992. Below this, there are tabs for DETECTION, DETAILS, and COMMUNITY. A banner encourages joining the community for more insights. A 'Popular threat label' section shows 'trojan.aaaaa/mark'. The 'Security vendors' analysis table lists 18 detections from various vendors, including ALYac, Avira, ClamAV, Cynet, eScan, QuickHeal, Symantec, TrendMicro, and VIPRE, each with a specific detection name like 'Gen:Trojan.FileInfector.aaW@aaaaa' or 'Trojan.FileInfector.ED11F2'. A 'Threat categories' section shows 'trojan' and 'virus', and a 'Family labels' section shows 'aaaaa', 'mark', and 'unlv'. A link to 'Join our Community' is also present.

18 / 62
Community Score

18/62 security vendors flagged this file as malicious

Reanalyze Similar More

e178f653f95b87cd7ed2eb5124a0d11520ecac132c1eacf22e6b7a3efee263e3
1992.EXE
mz

Size: 924 B | Last Analysis Date: 12 days ago

DETECTION DETAILS COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to [automate checks](#).

Popular threat label: trojan.aaaaa/mark

Threat categories: trojan virus

Family labels: aaaaa mark unlv

Security vendors' analysis

ALYac	Gen:Trojan.FileInfector.aaW@aaaaa	Arcabit	Trojan.FileInfector.ED11F2
Avira (no cloud)	Mark II	BitDefender	Gen:Trojan.FileInfector.aaW@aaaaa
ClamAV	Win.Trojan.MarkII-1	CTX	Unknown.trojan.fileinfector
Cynet	Malicious (score: 99)	Emsisoft	Gen:Trojan.FileInfector.aaW@aaaaa (B)
eScan	Gen:Trojan.FileInfector.aaW@aaaaa	GData	Gen:Trojan.FileInfector.aaW@aaaaa
QuickHeal	Search.350	Skyhigh (SWG)	Univ/a
Symantec	Mark II	Trellix ENS	Univ/a
TrendMicro	MARK_2	TrendMicro-HouseCall	MARK_2
VIPRE	Gen:Trojan.FileInfector.aaW@aaaaa	WithSecure	Malware.Mark II

Do you want to automate checks?

1992.EXE / 924 bytes / identified as MZ / 18 of 62

The submitted objects differed

Submitted file	Recognized format	Reported detections
1992code.com 412 bytes	gzip	0 / 61
1992.EXE 924 bytes	MZ	18 / 62

Captured January 22, 2026. Different hashes, formats and analysis ages.

A green result leaves questions open

**0 detections
≠ proof of safety**

File identity and execution evidence still matter.

Windows 7 kept **.com** command names

Examples in C:\Windows\System32



CONSOLE

Text-mode utilities

chcp.com — Console code page

mode.com — Device and port settings

more.com — Output one screen at a time



DISKS

Floppy disk utilities

diskcomp.com — Compare floppy disks

diskcopy.com — Copy floppy disks

format.com — Format a volume



DIRECTORIES

Directory information

tree.com — Display the folder hierarchy

A **.com** extension does not establish a DOS binary format.

Windows PE executables can retain .com names. This does not show that the DOS infector works on Windows 7.

Why didn't I just use AI??!!??

I DID!

The OpenAI models gave me a “nastygram.”

THEN DEBUGGING

Every OpenAI model I tried struggled.

ANTHROPIC

Worked better and finished the task.

OPUS

Two iterations. Seriously.

Embrace the future. It's here.

My cleaner: disinfect.asm

Restore the entry.

Remove the appendage.

A separate DOS tool works on the file on disk.

This trace belongs to the variant

```
A:\>find
```

```
US GB FF FO FH RD CL RE EX
```

```
A:\>_
```

RD: read / CL: close / RE: restore entry / EX: exit

A jump byte is a weak detector

CURRENT TEST

First byte is E9

LIMITATION

Legitimate .COM files
can start with a jump

A candidate match alone cannot justify rewriting a file.

Recovery deserves a before-and-after test

ORIGINAL

Preserved file and hash

CLEANED

Byte comparison and expected behavior

CONTROLS

Uninfected and unsupported files

Proposed demonstration evidence, not claimed measurements.



Mangia baby!!

Modern parallels: evidence survives the era

Historical observation	Modern question	Evidence to seek
Execution path changes	Who altered execution?	Provenance and state difference
Marker in a file	What does it establish?	Content plus context
Host behavior resumes	What happened first?	Event sequence and process lineage

What the reconstruction taught me

SOURCE

A plausible transcription can still be wrong

LAB

A visible stage is stronger than a guess

DEFENSE

Cleaning needs evidence of recovery

RESPECT

The old-school coders knew their stuff.

Rebuilt. Inspected. Detected. Cleaned.

**Old code.
Very current questions.**

Charley Pfaff
Questions

